

Python List [append() and insert()]- Internal Working Explained

1. What is a List in Python?

A list is a built-in Python data structure used to store multiple items in a single variable. Lists are ordered, mutable (changeable), and can contain elements of different data types. Python implements a list as a dynamic array that stores references to objects.

2. append() Operation

Syntax: list_name.**append**(value)

Function: Adds an element at the end of the list.

WHAT HAPPENS IN BACKGROUND:

1. Python checks whether the list has free capacity.
2. If space is available, the new element's reference is placed in the next free slot.
3. The list size increases by 1.
4. If space is not available, Python allocates a larger memory block.
5. Existing element references are copied to the new block.
6. The new element is added at the end.
7. The old memory block is released.

Example:

```
L = [10, 20, 30]
```

```
L.append(40)
```

```
Result: [10, 20, 30, 40]
```

3. insert() Operation

Syntax: list_name.insert(index, value)

Function: Inserts an element at a specific position.

Background Steps:

Suppose:

L = [10, 20, 30, 40]

L.insert(1, 15)

1. Python checks whether enough capacity exists.
2. If needed, the list is resized.
3. Elements from the insertion position onward are shifted one position to the right.
4. The new element's reference is placed in the empty slot.
5. The list size increases by 1.

Before insertion:

Index: 0 1 2 3

Value: 10 20 30 40

After shifting:

Index: 0 1 2 3 4

Value: 10 _ 20 30 40

After inserting 15:

Value: 10 15 20 30 40

4. `append()` vs `insert()`

`append(value)`

- Adds at the end.
- Usually very fast.
- Average complexity $O(1)$.

`insert(index, value)`

- Adds at a specific position.
- Requires shifting elements.
- Complexity $O(n)$.

BRIEF DIFFERENCE BETWEEN APPEND AND INSERT

`append()`: Python places the new element at the end of the dynamic array. If memory is full, a larger block is allocated and elements are copied. Average complexity is $O(1)$.

`insert()`: Python creates space at the required index by shifting all next elements one position to the right and then inserts the new element. Complexity is $O(n)$.